

FORGE: Towards Functional Tool-Use Generalization via Keypoint Trajectory Reasoning

Jianfei Yang^{1,2}

¹MARS Lab, ²NTU

While humans readily repurpose a book, a stone, or a shoe to drive a nail, robots trained on specific tools fail to transfer the same function to novel ones – a gap we formalize as *functional generalization*. Such tools share a common functional intent that is visually recognizable, yet this perceptual similarity does not carry over to action space, where each tool demands an entirely different motor pattern. To bridge this gap, we explore intermediate representations including affordance images, human video prompts, and 2D keypoint trajectories, finding that keypoint trajectories best balance functional expressiveness and action groundability. Building on this, we propose **FunctiOnal Reasoning and Grounded Execution (FORGE)**, a two-stage policy that decouples functional reasoning from action execution: predicting generalizable keypoint trajectories from action-free data, then grounding them into robot actions with limited demonstrations. On a seven-tool hitting-function benchmark, FORGE consistently outperforms state-of-the-art methods on unseen tools in both simulation and the real world, achieving over $2\times$ improvement in average success rate. More experimental results are available at: <https://FORGE-policy.github.io>.

Correspondence: Jianfei Yang at jianfei.yang@ntu.edu.sg



1 Introduction

In open-ended real-world environments, the right tool is rarely at hand, yet humans adapt effortlessly, repurposing a book, a stone, or a shoe to drive a nail, because functionally equivalent tools share a common intent: a contact region to strike with and a motion to bring it onto the target. This capacity, which we term *functional generalization*, is a hallmark of human dexterity that robots have yet to achieve. Current manipulation policies overfit to the appearance and geometry of seen tools, failing entirely when handed a novel one that serves the same function [Chen et al. \(2025\)](#); [Turpin et al. \(2021\)](#); [Qin et al. \(2023\)](#).

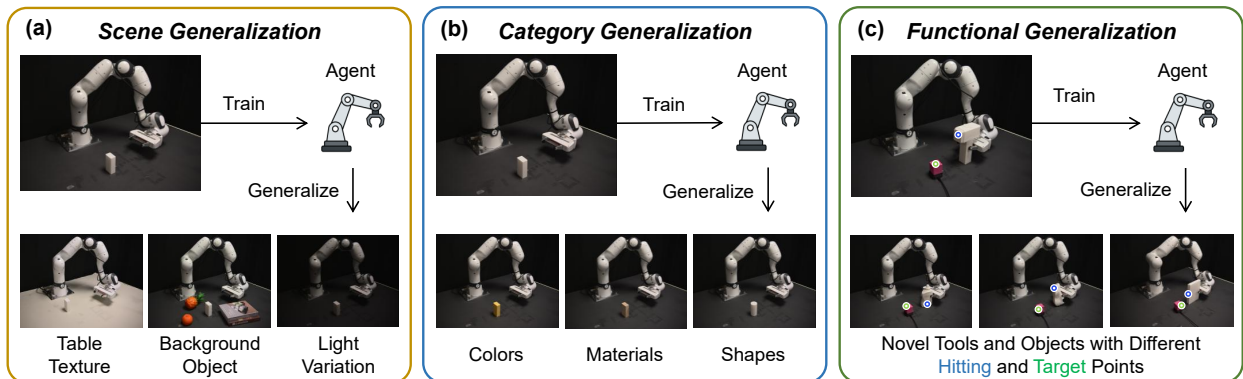


Figure 1 Comparison between functional generalization and existing generalization settings. (a) Scene generalization evaluates robustness to visual variations. (b) Category generalization tests transfer across objects with diverse properties. (c) Functional generalization requires using unseen tools to accomplish the same function.

Functional generalization is fundamentally harder than conventional generalization. As illustrated in Fig. 1, scene- and category-level generalization only require tolerating visual variations while the underlying motion

stays the same Goyal et al. (2023); Shridhar et al. (2023, 2022); Nair et al. (2023). Functional generalization, by contrast, demands that the motion itself change: striking a target with a novel tool requires locating its contact region, aligning it, and producing an appropriate motion, even when the tool’s shape and trajectory differ entirely from training. The core difficulty is a fundamental mismatch: functionally equivalent tools share recognizable structure in visual space, but this similarity does not transfer to action space, where each tool demands an entirely different motor pattern.

Bridging this perception-to-action gap requires an intermediate representation that carries functional intent across tools, satisfying two competing demands: expressive enough to capture where to make contact and how to move onto the target, yet grounded enough to be predicted from action-free observations without overfitting to tool-specific appearance. Dense representations like raw video entangle function-relevant cues with tool-specific appearance, while overly sparse ones discard the geometric and temporal structure needed for action. This trade-off raises the central question of our work: *which representation best transfers functional intent from perception to action?*

To answer this question, we systematically evaluate intermediate representations including affordance images, human video prompts, and 2D keypoint trajectories, finding that keypoint trajectories best balance functional expressiveness and action groundability: they capture function-relevant contact and motion structure while discarding tool-specific appearance that causes overfitting. Building on this, we propose **FunctiOnal Reasoning and Grounded Execution** (FORGE), a two-stage policy that decouples functional reasoning from action execution: it first predicts generalizable keypoint trajectories for unseen tools from large-scale action-free data, then grounds these functional plans into executable robot actions using a small set of action-labeled demonstrations. We further introduce a seven-tool hitting-function benchmark, where FORGE consistently outperforms state-of-the-art methods on unseen tools in both simulation and the real world, achieving over $2\times$ improvement in average success rate. In summary, our contributions are threefold:

- We formalize *functional generalization* in robotic tool-use and identify the perception-to-action gap as its core difficulty.
- We find that 2D keypoint trajectories best balance functional expressiveness and action groundability, and propose FORGE, a two-stage policy that predicts generalizable keypoint trajectories from action-free data and grounds them into robot actions with limited demonstrations.
- Through extensive simulation experiments across seven tools and real-world validation, we demonstrate that FORGE consistently outperforms state-of-the-art methods on unseen tools, achieving over $2\times$ improvement in average success rate.

2 Related Works

Generalization Problems in Robotic Manipulation. Generalization has been a central problem in robotic manipulation, and existing studies mainly evaluate it from three perspectives: scene-level, category-level, and cross-embodiment generalization. Scene-level generalization focuses on whether a policy can remain robust when the surrounding visual or physical contexts vary, such as changes in object layouts, lighting conditions, or camera viewpoints Goyal et al. (2023); Shridhar et al. (2023, 2022); Nair et al. (2023). Furthermore, category-level generalization studies whether robots can transfer manipulation skills to unseen objects that belong to the same semantic category while differing in geometry, size, or texture Gao and Tedrake (2021); Mo et al. (2021); Geng et al. (2023). More recently, cross-embodiment generalization has attracted increasing attention, which aims to train a unified policy that can transfer across different robot platforms, such as robotic arms and humanoids O’Neill et al. (2024); Team et al. (2024); Intelligence et al. (2025). In this work, we go beyond generalization over visual appearance and robot embodiment and focus on *functional generalization*, in which an agent is required to accomplish the same function using diverse, unseen tools.

Improving Generalization in Robot Policy Learning. To improve manipulation generalization, the most straightforward way is scaling up action-labeled robotic data and training policies end-to-end Zitkovich et al. (2023); O’Neill et al. (2024); Team et al. (2024); Khazatsky et al. (2024). However, collecting large-scale robotic data is time-consuming and labor-intensive, especially for complicated tool-use manipulations. Reducing the

dependence on action labels, some works learn transferable motion priors from large-scale action-free video data Nair et al. (2023); Wang et al. (2023); Wu et al. (2024); Ma et al. (2026); Pai et al. (2025); An et al. (2026); Hou et al. (2026). For example, MimicPlay Wang et al. (2023) learns high-level latent plans from human play videos and grounds them into robot actions with a low-level visuomotor policy. Although videos contain rich motion and interaction cues, dense future frames do not explicitly reveal a compact abstraction of function-relevant intent. As a result, effectively learning functional priors from videos often requires extremely extensive pretraining. Another line of work improves action grounding with more structured intermediate representations, such as interaction heatmaps Mo et al. (2021); Bahl et al. (2023); Wu et al. (2025); Li et al. (2026), object-centric representations Zhu et al. (2023); Chen et al. (2025); Chapin et al. (2025), or keypoint trajectories Gao and Tedrake (2021); Huang et al. (2025); Turpin et al. (2021); Wen et al. (2023). For instance, ATM Wen et al. (2023) predicts future trajectories of arbitrary points from video and uses them to guide a low-level policy for action generation. However, these methods mainly focus on standard manipulation tasks, such as object pick-up, while overlooking tool-use scenarios that require capturing more abstract functional intent, which are systematically investigated in our work.

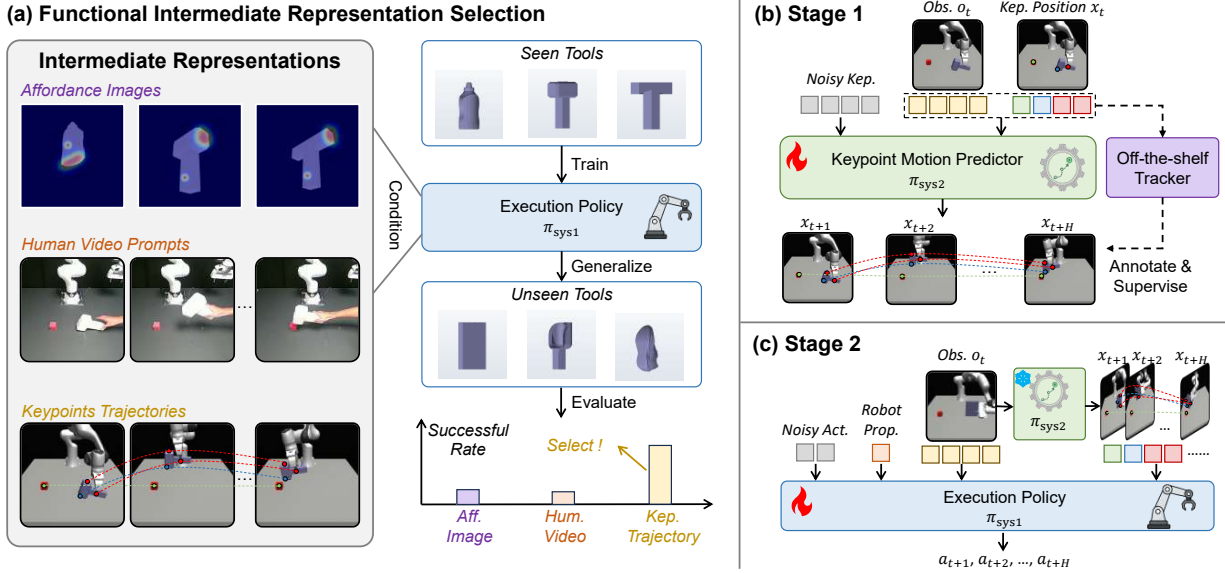


Figure 2 Overview of the FORGE. (a) Evaluation and selection of functional intermediate representations. (b) In Stage 1, the System-2 planner learns to predict future keypoint-based functional plans from action-free observations. (c) In Stage 2, the System-1 execution policy grounds the predicted functional plans into executable robot actions.

3 Methods

3.1 Problem Formulation

We formulate functional generalization in tool-use manipulation as a Markov Decision Process (MDP), where the state $\mathbf{s}_t$ consists of the visual observation $\mathbf{o}_t$, the proprioceptive state $\mathbf{q}_t$, and optionally a functional intermediate representation $\mathbf{x}_t$ (e.g., affordance images, human video prompts, or keypoint trajectories) that carries function-relevant cues bridging perception and action, and the action $\mathbf{a}_t$ is the motor command. A task function (e.g., hitting a target) is defined over seen tools $\mathcal{T}_{\text{seen}}$ for training and disjoint unseen tools $\mathcal{T}_{\text{unseen}}$ for evaluation, where the function stays fixed but the tool, its appearance, and the required contact positions and motions all change.

Two types of data are available for learning. The first is a large-scale action-free dataset $\mathcal{D}_U = \{(\mathbf{O}^{(i)}, \mathbf{X}^{(i)})\}_{i=1}^{N_U}$, consisting of visual observations $\mathbf{O}^{(i)} = \{\mathbf{o}_t^{(i)}\}_{t=1}^T$ and corresponding functional intermediate representations $\mathbf{X}^{(i)} = \{\mathbf{x}_t^{(i)}\}_{t=1}^T$, which is abundant since it requires no robot labels. The second is a smaller action-labeled robotic dataset $\mathcal{D}_L = \{(\mathbf{O}^{(i)}, \mathbf{X}^{(i)}, \mathbf{Q}^{(i)}, \mathbf{A}^{(i)})\}_{i=1}^{N_L}$, which additionally provides proprioceptive states $\mathbf{Q}^{(i)}$ and

ground-truth actions $\mathbf{A}^{(i)}$ but is costly to collect, so $|\mathcal{D}_L| \ll |\mathcal{D}_U|$. This data asymmetry, together with the perception-to-action gap discussed in Sec. 1, motivates a two-stage decomposition:

$$\underbrace{\pi_{\text{FORGE}}(\mathbf{a}_{t:t+H} \mid \mathbf{o}_t, \mathbf{q}_t, \mathbf{x}_t)}_{\text{FORGE}} = \underbrace{\pi_{\text{sys2}}(\hat{\mathbf{X}}_{t:t+H} \mid \mathbf{o}_t, \mathbf{x}_t)}_{\text{functional reasoning}} \underbrace{\pi_{\text{sys1}}(\mathbf{a}_{t:t+H} \mid \mathbf{o}_t, \mathbf{q}_t, \hat{\mathbf{X}}_{t:t+H})}_{\text{grounded execution}}. \quad (1)$$

FORGE learns a future predictor on $\mathcal{D}_U$ for forecasting functional intermediate representations from observations, and an execution policy on $\mathcal{D}_L$ for grounding predicted representations into actions.

3.2 Functional Intermediate Representations Selection

As discussed in Sec. 1, a desired functional intermediate representation $\mathbf{X}_{t:t+H}$ should preserve the function-relevant structure shared across tools, while remaining predictable from action-free observations and groundable into precise robot actions. We hypothesize that 2D keypoint trajectories form a suitable choice of $\mathbf{X}_{t:t+H}$. They discard tool-specific appearance that may cause overfitting, while retaining compact geometric information such as the hitting point, target position, and affordance-relevant tool structure. At the same time, they provide a structured temporal description of how function-relevant points should move, which makes them easy to predict from existing vision models Wen et al. (2023) and to ground into actions. To validate this hypothesis, we compare keypoint trajectories with two alternative choices of $\mathbf{X}_{t:t+H}$: affordance images and human video prompts.

Affordance Image. Given an affordance image $I_{\text{aff}}^{(i)}$, we encode it with an image encoder E_{aff} He et al. (2016) and use the resulting feature as a time-invariant condition: $x_t^{(i)} = E_{\text{aff}}(I_{\text{aff}}^{(i)}) \in \mathbb{R}^d$.

Human Video Prompt. Given a human demonstration video $V_{\text{hum}}^{(i)} = \{I_{\text{hum},t}^{(i)}\}_{t=1}^T$, we encode it with a video encoder E_{vid} Bertasius et al. (2021) and apply temporal average pooling to obtain a sequence-level feature, which is used as a time-invariant condition: $x_t^{(i)} = \text{AvgPool}(E_{\text{vid}}(V_{\text{hum}}^{(i)})) \in \mathbb{R}^d$.

Keypoint Trajectory. We denote the keypoint trajectory as $P^{(i)} = \{p_t^{(i)}\}_{t=1}^T$, where each frame $p_t^{(i)} = \{p_t^{(i),k}\}_{k=1}^K$ contains K 2D keypoints. At time step t , we flatten the keypoint coordinates and encode them with a keypoint encoder E_{kp} , yielding $x_t^{(i)} = E_{\text{kp}}(\text{Flatten}(p_t^{(i)})) \in \mathbb{R}^d$.

To evaluate these candidates, we design a heuristic experiment in Fig. 2 (a), where each representation is pre-extracted for both seen and unseen tools and directly used as the condition for the execution policy π_{sys1} . The policy is trained on action-labeled data from seen tools in $\mathcal{D}_L$ and evaluated on unseen tools, where success is determined in pixel space by checking whether the predicted hitting point overlaps with the target point within a tolerance threshold. Among the three candidates, 2D keypoint trajectories achieve the best performance, since they go beyond static region cues and dense visual motion by providing a compact, structured description of how function-relevant points move and align over time. We therefore adopt 2D keypoint trajectories as the functional intermediate representation for FORGE.

3.3 Functional Reasoning and Grounded Execution Policy

After selecting 2D keypoint trajectories as the functional intermediate representation for FORGE, we design a two-stage training strategy to decouple functional reasoning from action execution. This enables FORGE to learn transferable functional keypoint motion from action-free demonstrations, while grounding the predicted trajectories into robot actions with limited action-labeled data.

Stage 1: Action-free Functional Reasoning. As shown in Fig. 2 (b), we train the keypoint motion predictor π_{sys2} on the action-free dataset $\mathcal{D}_U$ to predict future functional keypoint trajectories. Given the visual observation $\mathbf{o}_t$ and keypoint representation $\mathbf{x}_t$ at time-step t , π_{sys2} generates a future keypoint trajectory $\hat{\mathbf{X}}_{t:t+H}$ over horizon H .

Formally, we instantiate π_{sys2} as a conditional flow-matching model in the keypoint space. Given a ground-truth keypoint trajectory $\mathbf{X}_{t:t+H}$, a Gaussian noise sample $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, and an interpolation time $\tau \sim \mathcal{U}(0, 1)$, we define the interpolated keypoint trajectory as $\mathbf{X}_{t:t+H}^\tau = \tau \mathbf{X}_{t:t+H} + (1 - \tau)\epsilon$. The keypoints motion predictor

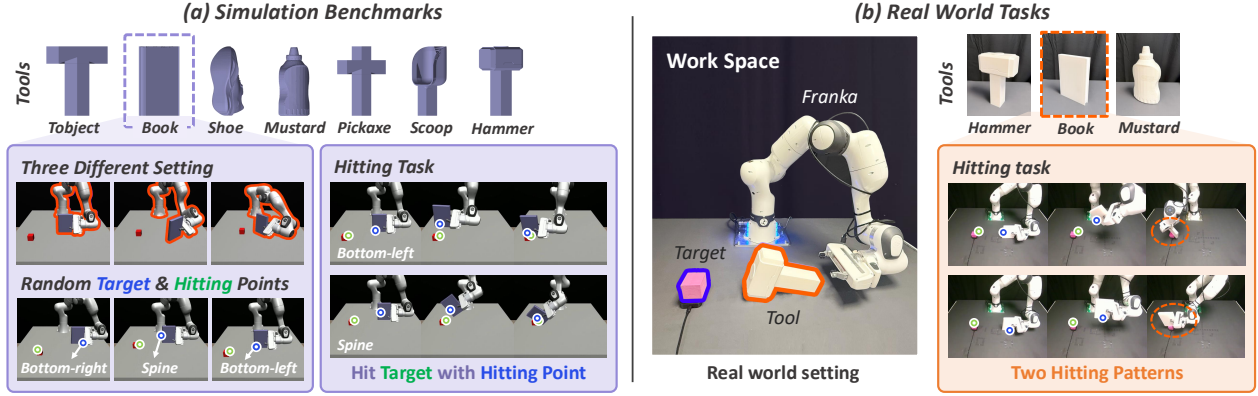


Figure 3 Simulation Benchmark and Real-world Setting. (a) The simulation benchmark contains seven tools, three initial settings per tool, and randomized target and hitting points. (b) The real-world setting uses a Franka robot and includes three tools, each with two hitting patterns.

learns a conditional velocity field u_ϕ by minimizing

$$\mathcal{L}_{\text{sys2}} = \mathbb{E}_{\tau, \epsilon, \mathcal{D}_U} \left[\left\| u_\phi \left(\mathbf{X}_{t:t+H}^\tau, \tau \mid \mathbf{o}_t, \mathbf{x}_t \right) - (\mathbf{X}_{t:t+H} - \epsilon) \right\|_2^2 \right]. \quad (2)$$

During training, our task specifies the hitting point on the tool and the target point on the object. Given the hitting point, we use SAM2 [Ravi et al. \(2025\)](#) to segment the corresponding tool mask and sample N keypoints on the mask with Farthest Point Sampling (FPS). As shown in Fig. 2 (b), we then track these keypoints with CoTracker [Karaev et al. \(2024\)](#) to obtain keypoint trajectories for training. Since this stage only requires visual observations and keypoint trajectories, System-2 can leverage action-free data to capture transferable functional intent as keypoint-based motions, without relying on large-scale robot action labels.

Stage 2: Grounded Execution Policy. In the second stage, we freeze the pretrained keypoint motion predictor and train the execution policy π_{sys1} on the action-labeled robotic dataset $\mathcal{D}_L$. Given the observation $\mathbf{o}_t$, robot state $\mathbf{q}_t$, and 2D keypoint pixels $\mathbf{x}_t$, the frozen π_{sys2} first predicts future keypoint motions:

$$\hat{\mathbf{X}}_{t:t+H} = \pi_{\text{sys2}}(\mathbf{o}_t, \mathbf{x}_t). \quad (3)$$

To improve robustness to prediction errors, we apply random pixel perturbations to the $\hat{\mathbf{X}}_{t:t+H}$ during training, where each keypoint is shifted by 1–5 pixels with a probability of 0.5. The execution policy π_{sys1} then grounds the perturbed keypoint trajectories $\tilde{\mathbf{X}}_{t:t+H}$ into executable robot actions:

$$\hat{\mathbf{A}}_{t:t+H} = \pi_{\text{sys1}}(\mathbf{o}_t, \mathbf{q}_t, \tilde{\mathbf{X}}_{t:t+H}). \quad (4)$$

Similar to π_{sys2} , we instantiate π_{sys1} as a conditional flow-matching model in the action space. Given a ground-truth action chunk $\mathbf{A}_{t:t+H}$, a Gaussian noise sample $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, and an interpolation time $\tau \sim \mathcal{U}(0, 1)$, we define the interpolated action trajectory as $\mathbf{A}_{t:t+H}^\tau = \tau \mathbf{A}_{t:t+H} + (1 - \tau)\epsilon$. The action-grounded execution policy learns a conditional velocity field v_θ by minimizing:

$$\mathcal{L}_{\text{sys1}} = \mathbb{E}_{\tau, \epsilon, \mathcal{D}_L} \left[\left\| v_\theta \left(\mathbf{A}_{t:t+H}^\tau, \tau \mid \mathbf{o}_t, \mathbf{q}_t, \tilde{\mathbf{X}}_{t:t+H} \right) - (\mathbf{A}_{t:t+H} - \epsilon) \right\|_2^2 \right]. \quad (5)$$

4 Experiments

Functional generalization remains a largely underexplored problem in robotic manipulation. In this work, we instantiate it with the hitting function and construct both simulation and real-world benchmarks to systematically evaluate whether a policy can transfer the same function to unseen tools. Specifically, we design experiments to test four hypotheses that together validate the central claims of this work:

- **H1:** Functional generalization requires an intermediate representation, not end-to-end mapping.
- **H2:** Among candidate intermediate representations, 2D keypoint trajectories best balance affordance expressiveness and action groundability.
- **H3:** Keypoint trajectories must be function-aware; generic keypoint tracking is not enough.
- **H4:** The two-stage design generalizes under moderate action-labeled tool diversity, making it a practical recipe for tool-use learning.

4.1 Experimental setup

Simulation Benchmark. As shown in Fig. 3, our simulation benchmark contains 7 tools, each with 3 different initial settings. For each tool-setting pair, we collect 30 demonstrations using a four-stage Model Predictive Path Integral (MPPI) planner, resulting in 630 demonstrations in total. The MPPI planner generates an action trajectory by optimizing stage-dependent objectives that sequentially encourage the robot to lift the tool, move it toward the target, align the tool-specific hitting point with the target cube, and execute the final strike. In each demonstration, we randomly initialize the target cube position and sample the tool-specific hitting point along the tool boundary, requiring the policy to adapt its motion to both the tool geometry and the target location. In the main setting, all methods are trained on 4 tools, including hammer, tobject, pickaxe, and mustard, and evaluated on the remaining 3 unseen tools. During evaluation, each unseen tool is tested under three initial settings with 30 rollout episodes. A rollout is considered successful only if the robot strikes the target cube using the designated hitting point.

Real-World Setting. We further construct a real-world benchmark on a Franka robot platform, as shown in Fig. 3. The real-world setting contains 3 tools: hammer, mustard, and book. Each tool has two hitting patterns, and each pattern contains 20 expert demonstrations, resulting in 120 real-world trajectories. We train the policy on hammer and mustard and evaluate its functional generalization ability on the unseen book tool. For each hitting pattern, we perform 20 real-world rollout trials. A trial is successful if the robot follows the specified hitting pattern and strikes the target.

Baselines. We compare FORGE against three baselines covering end-to-end and keypoint-based policies. *Flow-Matching (FM)* Lipman et al. (2023) and *Diffusion Policy (DP)* Chi et al. (2025) are end-to-end visuomotor policies that map observations directly to actions, serving as references for the perception-to-action gap (Sec. 1). *ATM* Wen et al. (2023) also conditions on keypoint trajectories, but obtains them from a generic tracker pretrained on LIBERO-90 rather than a function-aware predictor, allowing us to test whether the gain comes from using keypoints or from predicting function-aware keypoint motion. All baselines receive the same inputs as FORGE: visual observations, proprioceptive states, and the initial hitting and target point positions.

Evaluation Metrics. All experiments report the success rate (SR) as the main evaluation metric, which is defined as the number of successful trials divided by the total number of rollout trials.

Table 1 Simulation Results. Success rate (SR) on three unseen tools. Each setting is evaluated with 30 rollouts, and setting-level SR, tool-level average SR, and overall average SR are reported.

Method	Scoop				Shoe				Book				Overall
	S01	S02	S03	Avg.	S01	S02	S03	Avg.	S01	S02	S03	Avg.	
FM Lipman et al. (2023)	0.00	0.03	0.17	0.07	0.00	0.07	0.03	0.03	0.13	0.17	0.17	0.16	0.09
DP Chi et al. (2025)	0.13	0.13	0.27	0.18	0.00	0.13	0.10	0.08	0.33	0.23	0.23	0.26	0.17
ATM Wen et al. (2023)	0.13	0.17	0.07	0.12	0.10	0.00	0.07	0.06	0.07	0.07	0.03	0.06	0.08
FORGE (Ours)	0.30	0.30	0.67	0.42	0.23	0.23	0.23	0.23	0.53	0.20	0.53	0.42	0.36

4.2 Main Results & Analysis

Simulation Results. As shown in Tab. 1, FM and DP achieve only 0.09 and 0.17 average SR on unseen tools, validating **H1**: without an intermediate representation exposing the contact region and its alignment with

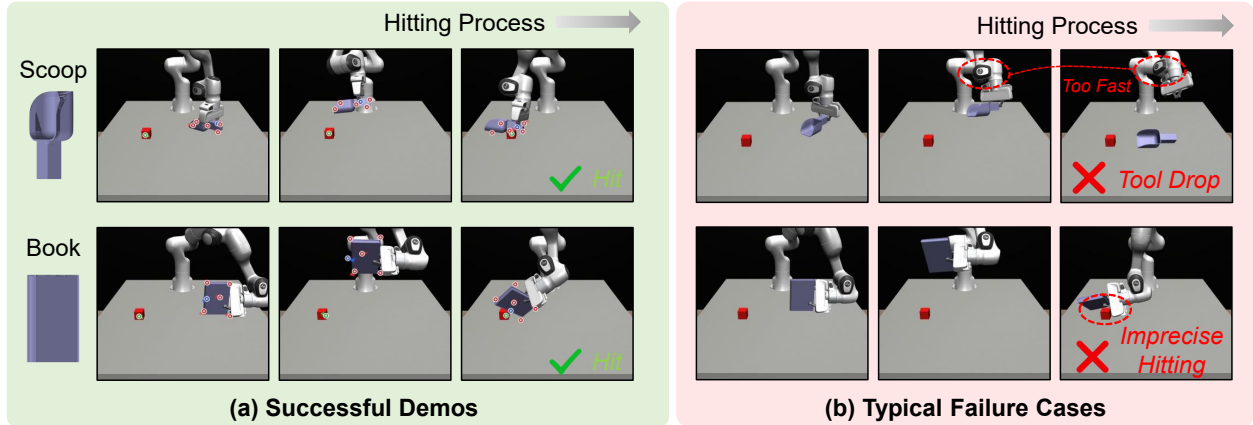


Figure 4 Qualitative results of FORGE. (a) Successful demos with predicted keypoint-based functional plans on unseen tools. (b) Two typical failure cases of FORGE.

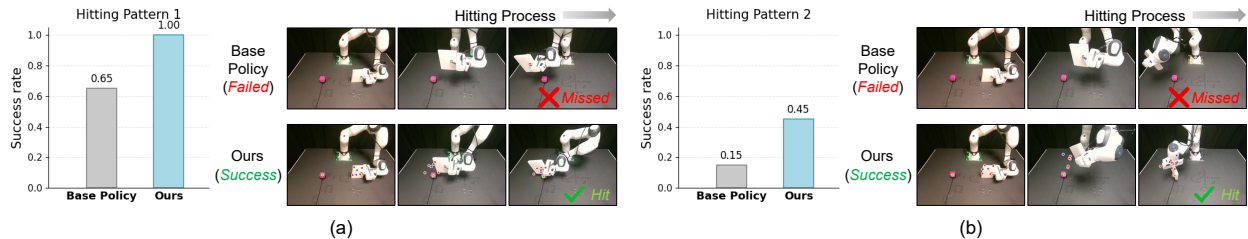


Figure 5 Real-World Results. Results on the unseen book across two hitting patterns. We compare FORGE with the FM base policy. Results for hitting with (a) the bottom. (b) the spine.

the target, end-to-end policies fall back on the motor patterns of the most visually similar seen tool. ATM also underperforms (0.08), validating **H3**: its generic tracker cannot anticipate function-relevant motion, so the keypoint trajectories it provides fail to describe how the contact region should approach the target. In contrast, FORGE achieves 0.36, outperforming DP by over $2\times$ with consistent gains across all three unseen tools, and Fig. 4 (a) shows that its predicted keypoint trajectories explicitly bring the tool-specific contact region toward the target before impact. We further observe two failure modes (Fig. 4 (b)): *imprecise hitting*, where the 2D trajectories capture the overall motion but lack fine-grained spatial alignment, motivating 3D-aware functional representations; and *tool dropping*, where the execution policy produces jerky actions, motivating smoothness regularization or trajectory post-processing.

Real-World Results. We further evaluate whether FORGE can transfer the learned hitting function to the real world by comparing it with the FM base policy on the unseen book tool. As shown in Fig. 5, FORGE consistently outperforms FM under both hitting patterns. This further supports **H1**, showing that direct perception-to-action mapping is insufficient for functional generalization. As illustrated by the failure cases in Fig. 5, FM often fails to align the correct hitting region of the unseen book tool with the target, leading to missed hits. In contrast, FORGE predicts function-relevant keypoint trajectories that specify how the desired hitting region should approach and contact the target. These intermediate plans provide explicit guidance for the grounded execution policy, enabling more accurate alignment and successful real-world hitting.

4.3 Ablation Studies

Beyond comparing against baselines, we conduct two ablations to dissect the design choices behind FORGE. The first ablation varies the functional intermediate representation, conditioning the execution policy on affordance images, human video prompts, or 2D keypoint trajectories, to identify which representation best balances affordance expressiveness and action groundability. The second ablation varies the diversity of

action-labeled tools, training FORGE on 1, 4, or 6 tools and testing on the unseen tools, to evaluate whether the two-stage pipeline provides a practical recipe for functional generalization using moderate robotic data.

4.3.1 Effect of Functional Intermediate Representations.

As discussed in Sec. 3.2, we investigate the effect of different functional intermediate representations by directly training the execution policy π_{sys1} . Results in Tab. 2 show that introducing affordance images or human video prompts improves generalization, suggesting that tool-region cues and visual motion demonstrations of the hitting provide useful functional information. In comparison, 2D keypoint trajectories capture fine-grained hitting and target point information while explicitly describing structured functional motion over time. These results support **H2**: keypoint trajectories provide a more compact and action-groundable condition, leading to the best functional generalization.

Table 2 Ablation Results on Functional Intermediate Representations. We condition FM on different representations and report the SR on 3 unseen tools.

Representation	Scoop	Shoe	Book	Avg.
Flow-Matching	0.07	0.03	0.16	0.09
+ Aff. Images	0.34	0.11	0.16	0.20
+ Hum. Vid. Prompts	0.26	0.09	0.23	0.19
+ Key. Trajectories	0.50	0.31	0.67	0.49

Table 3 Ablation on Action-Labeled Tool Diversity. Success rate (SR) of FORGE under different training-to-testing tool splits. ‘M-to-N’ denotes training on M tools and testing on N unseen tools.

Method	Tools							
	Hammer	Tobject	Pickaxe	Mustard	Scoop	Shoe	Book	Avg.
FORGE (1-to-6)	–	0.26	0.34	0.17	0.14	0.10	0.23	0.21
FORGE (4-to-3)	–	–	–	–	0.42	0.23	0.42	0.36
FORGE (6-to-1)	–	–	–	–	–	–	0.52	0.52

4.3.2 Effect of Action-Labeled Tool Diversity.

Beyond the standard 4-to-3 setting, we evaluate FORGE fine-tuned with robotic data from 1 or 6 tools and tested on the unseen ones. As shown in Tab. 3, the success rate grows monotonically with action-labeled tool diversity, from 0.21 (1-to-6) to 0.36 (4-to-3) to 0.52 (6-to-1), confirming that broader data coverage improves action grounding. This reveals a practical trade-off: too little robotic data limits grounding, while collecting demonstrations for many tools is costly. The non-trivial 4-to-3 performance supports **H4**: the two-stage design of FORGE provides a practical recipe, where scalable action-free data supports cross-tool functional reasoning and moderate action-labeled data suffices to ground predicted trajectories into executable actions.

5 Conclusion and Limitation

In this paper, we identify *functional generalization* as an important yet underexplored problem, where an agent needs to accomplish the same function with unseen tools. To address this problem, we propose FORGE, a policy that decouples functional reasoning from action execution via an intermediate representation. In this way, FORGE learns functional intent from action-free data to predict keypoint-based plans, and then grounds them into executable robot actions by fine-tuning on limited action-labeled demonstrations. We introduce a seven-tool hitting-function benchmark to evaluate functional generalization in simulation and real-world settings. Extensive experiments show that FORGE improves generalization to unseen tools over state-of-the-art methods.

Limitations and Future Work. First, the functional-relevant hitting and target points are specified in this study, while future work may leverage vision foundation models [Yuan et al. \(2025\)](#) or pretrain a keypoint proposal model to automatically identify function-relevant keypoints. Second, our pipeline does not consider

the grasping process. We will extend FORGE to predict grasp styles from keypoint-based functional trajectories and study more dedicated end-effectors such as dexterous hands.

References

- Tuo An, Jindou Jia, Gen Li, Jingliang Li, Chuhao Zhou, Pengfei Liu, Bofan Lyu, Jiaqi Bai, Xinying Guo, Geng Li, et al. Feedback world model enables precise guidance of diffusion policy. *arXiv preprint arXiv:2605.15705*, 2026.
- Shikhar Bahl, Russell Mendonca, Lili Chen, Unnat Jain, and Deepak Pathak. Affordances from human videos as a versatile representation for robotics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13778–13790, 2023.
- Gedas Bertasius, Heng Wang, and Lorenzo Torresani. Is space-time attention all you need for video understanding? In *Proceedings of the International Conference on Machine Learning*, 2021.
- Alexandre Chapin, Bruno Machado, Emmanuel Dellandrea, and Liming Chen. Object-centric representations improve policy generalization in robot manipulation. *arXiv preprint arXiv:2505.11563*, 2025.
- Haonan Chen, Cheng Zhu, Yunzhu Li, and Katherine Driggs-Campbell. Tool-as-interface: Learning robot policies from human tool usage through imitation learning. *arXiv e-prints*, pages arXiv–2504, 2025.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2025.
- Wei Gao and Russ Tedrake. kpm 2.0: Feedback control for category-level robotic manipulation. *IEEE Robotics and Automation Letters*, 2021.
- Haoran Geng, Helin Xu, Chengyang Zhao, Chao Xu, Li Yi, Siyuan Huang, and He Wang. Gapartnet: Cross-category domain-generalizable object perception and manipulation via generalizable and actionable parts. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023.
- Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. Rvt: Robotic view transformer for 3d object manipulation. In *Conference on Robot Learning*, 2023.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.
- Bohan Hou, Gen Li, Jindou Jia, Tuo An, Xinying Guo, Sicong Leng, Haoran Geng, Yanjie Ze, Tatsuya Harada, Philip Torr, et al. World model for robot learning: A comprehensive survey. *arXiv preprint arXiv:2605.00080*, 2026.
- Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In *Conference on Robot Learning*, 2025.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. pi0.5: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Cotracker: It is better to track together. In *European conference on computer vision*, 2024.
- Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- Jingliang Li, Jindou Jia, Tuo An, Chuhao Zhou, Xiangyu Chen, Shilin Shan, Boyu Ma, Bofan Lyu, Gen Li, and Jianfei Yang. Compassad: Intent-driven 3d affordance grounding in functionally competing objects. *arXiv preprint arXiv:2604.02060*, 2026.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- Teli Ma, Jia Zheng, Zifan Wang, Chunli Jiang, Andy Cui, Junwei Liang, and Shuo Yang. Dit4dit: Jointly modeling video dynamics and actions for generalizable robot control. *arXiv preprint arXiv:2603.10448*, 2026.
- Kaichun Mo, Leonidas J Guibas, Mustafa Mukadam, Abhinav Gupta, and Shubham Tulsiani. Where2act: From pixels to actions for articulated 3d objects. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.
- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. In *Conference on Robot Learning*, 2023.

- Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- Jonas Pai, Liam Achenbach, Victoriano Montesinos, Benedek Forrai, Oier Mees, and Elvis Nava. mimic-video: Video-action models for generalizable robot control beyond vlas. *arXiv preprint arXiv:2512.15692*, 2025.
- Meiyang Qin, Jake Brawer, and Brian Scassellati. Robot tool use: A survey. *Frontiers in Robotics and AI*, 2023.
- Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025.
- Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, 2022.
- Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, 2023.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- Dylan Turpin, Liquan Wang, Stavros Tsogkas, Sven Dickinson, and Animesh Garg. Gift: Generalizable interaction-aware functional tool affordances without labels. *Robotics: Science and Systems*, 2021.
- Chen Wang, Linxi Fan, Jiankai Sun, Ruohan Zhang, Li Fei-Fei, Danfei Xu, Yuke Zhu, and Anima Anandkumar. Mimicplay: Long-horizon imitation learning by watching human play. In *Conference on Robot Learning*, 2023.
- Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. *arXiv preprint arXiv:2401.00025*, 2023.
- Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, 2024.
- Shijie Wu, Yihang Zhu, Yunao Huang, Kaizhen Zhu, Jiayuan Gu, Jingyi Yu, Ye Shi, and Jingya Wang. Afforddp: Generalizable diffusion policy with transferable affordance. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 6971–6980, 2025.
- Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction in robotics. In *Conference on Robot Learning*, 2025.
- Yifeng Zhu, Zhenyu Jiang, Peter Stone, and Yuke Zhu. Learning generalizable manipulation policies with object-centric 3d representations. In *Conference on Robot Learning*, 2023.
- Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

Appendix

A Hitting Benchmark Construction and Implementation Details

[A.1](#) Extraction of Functional Intermediate Representations

[A.2](#) MPPI-based Demonstration Generation

[A.3](#) Training and Implementation Details

B Additional Experimental Results

[B.1](#) More Real-World Experiments

[B.2](#) Additional Qualitative Comparisons

A Hitting Benchmark Construction and Implementation Details

This section provides additional details on the construction of our hitting benchmarks. We first describe the extraction of different functional intermediate representations in Appx. [A.1](#), then introduce MPPI-based action-label generation in Appx. [A.2](#). Finally, we summarize the training and implementation details of FORGE and all baselines in Appx. [A.3](#).

Table 4 Statistics of Functional Intermediate Representations. We report the number of tools, settings, initial poses, samples per pose, and total samples for each representation.

Representation	# Tools	# Settings	# Initial Poses	# Samples / Pose	# Total
Affordance Image	7	1	1	1	7
Human Video Prompts	7	3	5	5	525
Keypoint Trajectories (Simulation)	7	3	30	1	630
Keypoint Trajectories (Real World)	6	2	1	20	240

A.1 Extraction of Functional Intermediate Representations

As discussed in Sec. [3.2](#), we consider three types of functional intermediate representations: affordance images, human video prompts, and 2D keypoint trajectories. Specifically, we describe how each representation is extracted and incorporated as an additional condition for the grounded execution policy π_{sys1} . The statistics of these representations are summarized in Tab. [4](#).

Affordance Images. We manually annotate the grasp point and hitting region for each tool, producing 7 affordance images in total. When evaluating functional intermediate representations in Sec. [3.2](#), we use the affordance image of the corresponding tool as an additional condition for the grounded execution policy π_{sys1} . Since this representation is time-invariant, the same image feature is shared across all time steps of a trajectory.

Human Video Prompts. As shown in Fig. [6](#), we collect human video prompts for all 7 tools. Following the simulation benchmark, each tool has 3 settings. For each setting, we define 5 initial positions and collect 5 videos per position, resulting in 525 human tool-use videos in total. When evaluating functional intermediate representations, we use the video prompt corresponding to the same tool, setting, and sample from Position 01 as an additional condition for the grounded execution policy. These videos provide visual demonstrations of how humans accomplish the same hitting function with different tools. We leave further exploitation of human video prompts, such as directly learning functional generalization from human tool-use demonstrations, to future work.

Keypoint Trajectories. We adopt 2D keypoint trajectories as the final functional intermediate representation and extract them for both simulation and real-world data. In simulation, the hitting and target points are directly obtained from the simulator. Given the initial hitting point on the tool, we use SAM2 [Ravi et al. \(2025\)](#) to segment the corresponding tool mask and sample $N = 5$ additional tool keypoints with Farthest

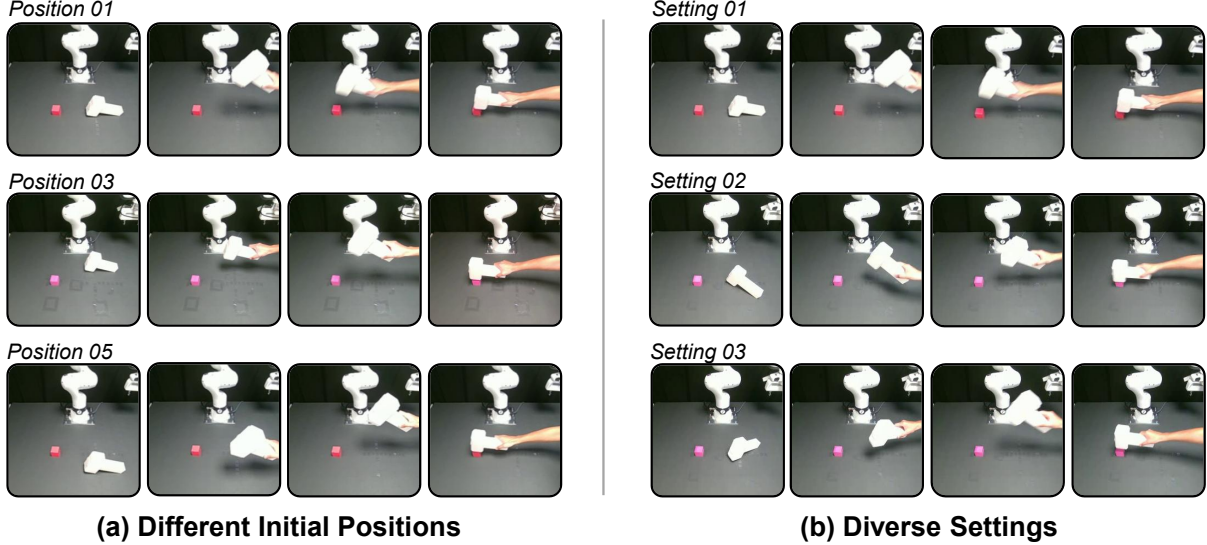


Figure 6 Human Video Prompt Samples. (a) Samples from different initial positions. (b) Samples from different settings.

Point Sampling (FPS), ensuring spatial coverage of the tool geometry. We then track these sampled tool keypoints with CoTracker [Karaev et al. \(2024\)](#) to obtain their 2D trajectories. Our simulation benchmark contains 7 tools with 3 settings each. For each setting, we randomly initialize 30 hitting and target positions, resulting in 630 simulation samples. For real-world data, we manually annotate the target and hitting points for each sample and obtain five additional tool keypoints using the same SAM2- and FPS-based procedure. Unlike simulation, where the hitting and target trajectories are available from the simulator, all 7 real-world keypoints are tracked by CoTracker. The real-world benchmark contains 6 tools, each with 2 hitting patterns and 1 initial position. We collect 20 samples for each initial position, resulting in 240 real-world samples.

A.2 MPPI-based Demonstration Generation

In the simulation benchmark, the action-labeled demonstrations are automatically generated rather than collected through teleoperation. We use a Model Predictive Path Integral (MPPI) planner with a four-stage rollout procedure, whose main reward terms are summarized in Tab. 5. The stages sequentially encourage the robot to lift the tool, move it toward the target, align the tool-specific hitting point with the target, and execute the final downward strike.

Besides the translational commands optimized by MPPI, we specify a target rotational pose at the end of each stage and generate the corresponding rotational commands through servo control. This helps the tool maintain an appropriate orientation during lifting, alignment, and hitting. Together, the stage-specific MPPI rewards and rotational servo control enable reliable automatic demonstration generation in simulation.

A.3 Training and Implementation Details

All models are trained on NVIDIA RTX 6000 Pro GPUs. For all methods, we use an action chunk size of 8, a history horizon of 8, and a batch size of 64. We build the simulation benchmark on Robomimic and conduct real-world experiments on a Franka robot platform.

For baseline methods, we train each policy for 120,000 steps using the corresponding action-labeled data. For FORGE, we follow the two-stage training procedure described in Sec. 3.3. We first pretrain the keypoint motion predictor $\pi_{\text{sys}2}$ on action-free data from all tools for 240,000 steps. We then freeze the pretrained predictor and fine-tune the grounded execution policy $\pi_{\text{sys}1}$ on the same action-labeled data used by the baselines for 120,000 steps.



Figure 7 Additional Real-World Qualitative Results. (a) Failure cases of the FM baseline on unseen tools. (b) Successful demos of FORGE across different hitting patterns. The blue point indicates the reference hitting point for each pattern.

B Additional Experimental Results

In this section, we first present additional real-world experiments with diverse unseen tools and hitting patterns in Appx. B.1, and then provide more qualitative analysis in Appx. B.2.

B.1 More Real-World Experiments

We conduct additional real-world experiments to evaluate FORGE under more diverse tool and hitting-pattern variations. Specifically, we train all methods on three seen tools, including hammer, pickaxe, and mustard, and evaluate them on three unseen tools: tobject, book, and shoe. Each unseen tool contains two hitting patterns, and each pattern is evaluated with 10 real-world rollout trials.

As shown in Tab. 6, FORGE outperforms the FM baseline across all three unseen tools, improving the overall success rate from 0.22 to 0.63. These results further confirm the main real-world finding: direct perception-to-action mapping struggles to adapt the hitting motion to novel tools, while keypoint-based functional plans provide explicit guidance for aligning the desired hitting region with the target. The FM

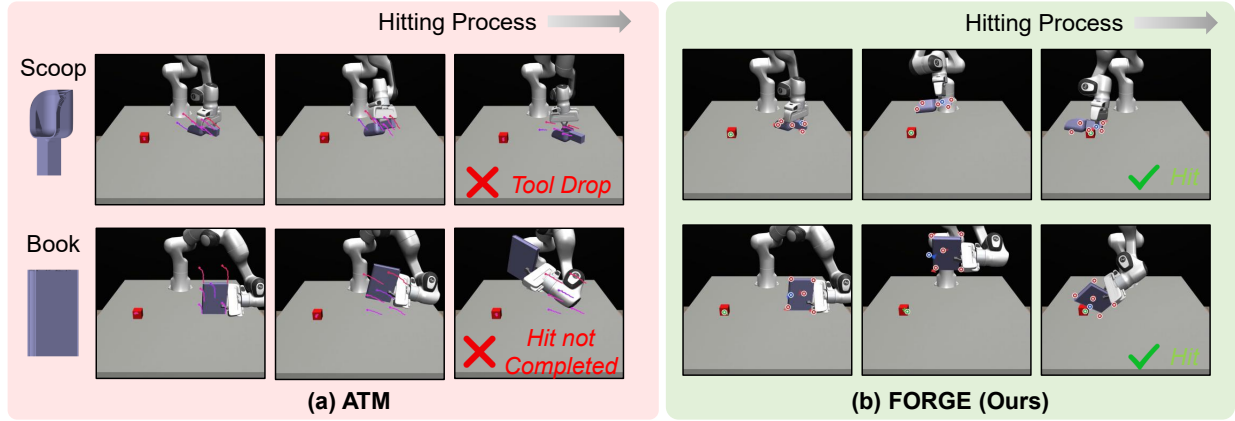


Figure 8 Additional Qualitative Comparison with ATM. (a) Failure cases of ATM on unseen tools. (b) Successful demos of FORGE with predicted keypoint-based functional plans.

baseline mainly fails due to ineffective downward strikes, missed hits, or incorrect hitting patterns. Please refer to our supplementary videos for qualitative demonstrations.

B.2 Additional Qualitative Comparisons

More Real-World Cases. Fig. 7 provides additional real-world comparisons between the FM baseline and FORGE across different unseen tools and hitting patterns. The FM baseline exhibits three typical failure modes: ineffective downward strikes, missed hits, and incorrect hitting patterns. These failures show that an end-to-end policy cannot reliably infer which tool region should be used or how the motion should change for a novel tool. By contrast, FORGE uses predicted keypoint-based functional plans to guide contact-region alignment and motion execution, leading to successful hits across these cases. This further supports that functional generalization benefits from an explicit intermediate representation between perception and action.

Comparison with ATM. Fig. 8 shows additional qualitative comparisons between ATM and FORGE on unseen tools. Although ATM also conditions the policy on keypoint trajectories, these trajectories are produced by a generic tracker and do not explicitly encode function-aware hitting motion. As a result, ATM often fails to guide the tool-specific contact region toward the target, leading to tool dropping or incomplete hits. In contrast, FORGE predicts function-aware keypoint trajectories that specify how the hitting point should approach the target before impact, enabling more reliable execution. These results further support our **H3** in Sec. 4: keypoint trajectories must encode functional motion intent rather than only generic point tracking.

Please refer to our supplementary videos for demos of both simulation and real-world experiments.

Table 5 Main Reward Terms for MPPI-based Demonstration Generation. Here, d_x , d_y , and d_{xy} denote the distances between the hitting point and the target along the x axis, the y axis, and the xy plane, respectively, while Δd_x , Δd_y , and Δd_{xy} denote their step-wise reductions. For the vertical direction, z_{lift} denotes the tool lifting height from its initial height, z_1 denotes the target lifting threshold in Stage 1, Δz_{lift} denotes the step-wise lifting progress, and e_z denotes the error between the current tool height and the target hovering height. Δz_{head} denotes the vertical displacement of the hitting point between consecutive steps, so $-\Delta z_{\text{head}}$ measures downward motion. The terms r_* reward the current state for satisfying the stage-specific objective, while r_{*_prog} rewards step-wise progress toward that objective.

Stage	Objective	Main Reward Terms
Stage 1	Lift the tool and align it with the target along the x and y axes.	$r_{\text{lift}} = 18.0 \min(z_{\text{lift}}, z_1),$
		$r_{\text{lift_prog}} = 100.0 \min(\Delta z_{\text{lift}}, 0.02),$
		$r_x = 22.0 \exp(-18.0 d_x),$
		$r_{x_prog} = 120.0 \min(\Delta d_x, 0.02),$
		$r_y = 8.0 \exp(-8.0 d_y),$
Stage 2	Move the tool toward the target while maintaining a suitable hovering height.	$r_{y_prog} = 35.0 \min(\Delta d_y, 0.02).$
		$r_{xy} = 14.0 \exp(-10.0 d_{xy}),$
		$r_x = 12.0 \exp(-18.0 d_x),$
		$r_y = 8.0 \exp(-10.0 d_y),$
		$r_z = 7.0 \exp(-18.0 e_z),$
Stage 3	Finely align the tool-specific hitting point above the target.	$r_{xy_prog} = 95.0 \min(\Delta d_{xy}, 0.02),$
		$r_{x_prog} = 80.0 \min(\Delta d_x, 0.02),$
		$r_{y_prog} = 40.0 \min(\Delta d_y, 0.02).$
		$r_{xy} = 14.0 \exp(-14.0 d_{xy}),$
		$r_x = 14.0 \exp(-24.0 d_x),$
Stage 4	Keep the hitting point close to the target and execute the downward strike.	$r_y = 10.0 \exp(-18.0 d_y),$
		$r_{xy_prog} = 145.0 \min(\Delta d_{xy}, 0.018),$
		$r_{x_prog} = 135.0 \min(\Delta d_x, 0.018),$
		$r_{y_prog} = 110.0 \min(\Delta d_y, 0.018),$
		$r_z = 7.0 \exp(-18.0 e_z).$
Stage 4	Keep the hitting point close to the target and execute the downward strike.	$r_{xy} = 12.0 \exp(-16.0 d_{xy}),$
		$r_x = 14.0 \exp(-28.0 d_x),$
		$r_y = 10.0 \exp(-18.0 d_y),$
		$r_{xy_prog} = 120.0 \min(\Delta d_{xy}, 0.015),$
		$r_{x_prog} = 100.0 \min(\Delta d_x, 0.015),$
Stage 4	Keep the hitting point close to the target and execute the downward strike.	$r_{y_prog} = 80.0 \min(\Delta d_y, 0.015),$
		$r_{\text{hit}} = 150.0 \min(-\Delta z_{\text{head}}, 0.05),$
Contact / Success	Encourage valid downward contact with the target.	$r_{\text{contact}} = 150.0$ if valid downward contact.

Table 6 Additional Real-World Results. Success rate (SR) on three unseen tools. Each tool is evaluated under two hitting patterns with 10 rollouts per pattern.

Method	Tobject			Book			Shoe			Overall
	P01	P02	Avg.	P01	P02	Avg.	P01	P02	Avg.	
FM Lipman et al. (2023)	0.00	0.60	0.30	0.50	0.00	0.25	0.00	0.20	0.10	0.22
FORGE (Ours)	0.60	0.80	0.70	0.60	1.00	0.80	0.20	0.60	0.40	0.63